Received: September 4, 2025. Revised: October 14, 2025.

INASS

International Journal of
Intelligent Engineering & Systems

http://www.inass.org/

Intelligent Site Reliability Engineering: A Multi-agent LLM Framework for Automated Incident Analysis and Root Cause Determination

Vipin Kataria[1]*

[1]*Picarro Inc, USA*
* Corresponding author's Email: vipink2@illinois.edu

Abstract: Modern distributed systems generate thousands of daily alerts that overwhelm Site Reliability Engineering (SRE) teams, causing alert fatigue, delayed incident response, and operational knowledge fragmentation. Traditional monitoring approaches using threshold-based alerting and manual correlation achieve only 45-65% accuracy in root cause identification and cannot scale with cloud-native microservices complexity. This paper presents an Intelligent SRE Agent employing a novel multi-agent Large Language Model (LLM) framework for automated incident analysis, root cause determination, and diagnostic reporting. The system implements a hierarchical architecture where a Master LLM Agent coordinates five domain-specialized agents via Model Context Protocol (MCP): Logs Agent for error pattern recognition, Metrics Agent for time-series anomaly detection, Traces Agent for distributed request flow analysis, Infrastructure Agent for platform resource assessment, and ML Services Agent for advanced pattern recognition. This multi-agent approach addresses single-LLM limitations by providing specialized domain expertise across heterogeneous operational data sources. We conducted comprehensive evaluation using dual testbeds: a controlled 7-microservice IoT sensor network with 150 systematically injected failure scenarios over 8 weeks, and an extended 50+ microservice production-replica environment with 200 scenarios. Chaos engineering tools including Toxiproxy, Chaos Monkey, and Litmus Chaos generated realistic failure modes encompassing cascading failures, resource exhaustion, configuration drift, and performance degradations. The Intelligent SRE Agent achieved 92.1% root cause identification accuracy compared to 67.3% manual baseline, reduced mean time to diagnosis by 82% (from 47.2 to 8.5 minutes), and demonstrated 96.3% precision in historical incident matching. Performance remained robust at scale, maintaining 89.4% accuracy in extended validation while handling complex enterprise-grade failures including multi-zone network partitions and cascading service dependencies.

Keywords: Site reliability engineering, Large language models, Incident management, Root cause analysis, Multi-agent systems, Chaos engineering.

1. Introduction

The exponential growth in complexity of modern microservices architectures has significantly increased the operational burden on Site Reliability Engineering (SRE) teams, who are tasked with managing the intricate web of dependencies and interactions inherent in these systems [1, 2]. Industry studies reveal that SRE teams spend 60-80% of their time on manual incident investigation and response, with large-scale distributed systems generating 10,000-50,000 alerts daily across monitoring tools

[3, 4]. This alert volume leads to critical alert fatigue, where engineers become desensitized to notifications, resulting in delayed incident response times that increase mean time to resolution (MTTR) from typical 30-45 minutes to over 2-3 hours for complex distributed system failures [5]. The operational knowledge required to manage these systems becomes increasingly fragmented as SRE teams must navigate vast arrays of observability data points across metrics, logs, and traces to diagnose issues effectively, with incident resolution requiring correlation across 15-25 different monitoring and observability tools [1, 6].

Traditional monitoring and incident response systems in distributed environments face several critical limitations that current approaches cannot

This article is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.
License details: https://creativecommons.org/licenses/by-sa/4.0/

adequately address. Threshold-based alerting systems generate false positive rates of 60-85% in dynamic cloud environments due to their inability to adapt to auto-scaling, seasonal patterns, and normal operational variance [7, 8]. Manual correlation processes do not scale beyond 100-200 microservices, requiring expert engineers to manually piece together evidence from disparate data sources, leading to investigation times that grow exponentially with system complexity [9, 10]. Even advanced AIOps platforms using traditional machine learning struggle with the contextual reasoning required for incident diagnosis, achieving only 45-65% accuracy in root cause identification due to their reliance on pattern matching without semantic understanding of system behavior and operational context [11]. These limitations collectively contribute to delayed root cause analysis, inconsistent incident resolution quality, and increased MTTR that directly impacts system reliability and business operations [9, 11].

Large Language Models (LLMs) present a transformative opportunity in operational intelligence by enabling semantic reasoning over unstructured operational data, natural language processing of log messages, and contextual understanding of incident patterns that traditional ML approaches cannot achieve [12]. Recent advances demonstrate LLMs' capability to process multimodal operational datasets, with systems achieving 75-85% accuracy in automated root cause analysis for cloud services through integration of logs, metrics, and configuration data [13]. However, existing LLM applications in operations typically employ single-model approaches that struggle with the specialized domain knowledge required across different operational data sources - logs require natural language processing expertise, metrics need statistical analysis capabilities, traces demand distributed systems knowledge, and infrastructure monitoring requires cloud platform understanding [12, 24, 25]. The complexity and volume of modern distributed system operations necessitate a more sophisticated approach that can dynamically leverage specialized analytical capabilities while maintaining the semantic reasoning advantages of LLMs.

This paper addresses these challenges by presenting an Intelligent SRE Agent that employs a novel multi-agent architecture where a Master LLM Agent orchestrates specialized domain-specific analytical agents to provide comprehensive incident analysis capabilities. The multi-agent approach is essential because different operational data sources require distinct analytical expertise - log analysis needs natural language processing for error message interpretation, metrics analysis requires statistical anomaly detection, trace analysis demands understanding of distributed request flows, and infrastructure monitoring needs cloud platform-specific knowledge. By specializing individual agents for specific domains while maintaining LLM-powered orchestration and reasoning, the system achieves both deep technical analysis and high-level strategic incident investigation that neither traditional monitoring nor single-LLM approaches can provide.

Our specific contributions include: (1) the first multi-agent LLM framework specifically designed for comprehensive incident analysis in distributed systems, with specialized agents for logs, metrics, traces, infrastructure, and ML analytics; (2) novel orchestration architecture using Model Context Protocol (MCP) that enables dynamic agent coordination based on incident characteristics and investigation requirements; (3) comprehensive experimental validation across 150 controlled failure scenarios in a realistic IoT sensor network environment with seven microservices; (4) significant performance improvements demonstrating 92.1% diagnostic accuracy and 82% reduction in mean time to diagnosis compared to manual analysis; and (5) production-ready framework that maintains human oversight while providing automated incident investigation capabilities suitable for real-world SRE operations.

The remainder of this paper is organized as follows: Section 2 reviews related work in monitoring systems, AIOps, and multi-agent architectures. Section 3 presents the methodology and system architecture of the Intelligent SRE Agent. Section 4 describes the experimental setup and presents results. Section 5 concludes with implications and future directions.

2. Related work

2.1 Traditional monitoring and incident management

Traditional monitoring and incident management systems originated from classical tools like Nagios and Zabbix, which were designed for static, on-premises environments with predefined metrics and threshold-based alerting [14, 15]. These systems relied heavily on rule-based event correlation and manual incident response workflows, requiring extensive human intervention for incident diagnosis and resolution [16]. While effective for simple, predictable infrastructure environments, these approaches fundamentally cannot scale with the dynamic nature of cloud-native microservices architectures [1, 2].

This article is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.
License details: https://creativecommons.org/licenses/by-sa/4.0/

The core limitation of traditional systems lies in their static configuration approach and inability to adapt to the dynamic scaling behaviours inherent in modern distributed systems [17]. The fragmentation of monitoring data across different systems further compounds these challenges, making it difficult to achieve the holistic view of system health necessary for effective incident management in complex microservices environments [18]. This fragmentation directly contributes to the knowledge fragmentation and alert fatigue experienced by modern SRE teams [3-6], establishing the need for more intelligent correlation and analysis approaches.

2.2 AIOps and machine learning approaches

AIOps has evolved significantly by leveraging machine learning and AI to enhance system operations through automated anomaly detection, incident prediction, and intelligent event correlation [12]. Early AIOps platforms focused primarily on automating existing workflows and improving decision-making by integrating traditional AI techniques into IT Operations Management [12]. Advanced anomaly detection approaches have been developed using both classical machine learning techniques and quantum computing methods for log-based analysis [19], while AI-driven alert correlation systems have enhanced pattern recognition and automated response mechanisms beyond simple rule-based approaches [20].

Modern AIOps platforms employ sophisticated supervised and unsupervised learning techniques, including autoencoders and recurrent neural networks, to improve anomaly detection accuracy in high-availability systems [21]. However, these approaches face persistent challenges including data quality issues, model interpretability problems, and the fundamental limitation of adapting to rapidly evolving system patterns in dynamic microservices environments [22]. Traditional ML approaches achieve only 45-65% accuracy in root cause identification due to their reliance on statistical pattern matching without semantic understanding of system behavior and operational context [9, 10]. These limitations demonstrate the need for more advanced approaches that can perform semantic reasoning over operational data.

2.3 LLMs in system operations

The integration of Large Language Models into system operations represents a paradigm shift toward semantic understanding of operational data, addressing critical limitations of traditional pattern-matching approaches. LLMs have demonstrated transformative potential in processing unstructured operational data, with LogParser-LLM achieving high accuracy in log parsing without extensive hyperparameter tuning [24]. The SuperLog model, developed through continual pre-training with domain-specific operational knowledge, has established new benchmarks in log analysis tasks by understanding the semantic context of log messages [25].

LLM-based systems have shown promise in industrial automation settings through chatbot interfaces that improve user interactions and automate routine operational queries [26]. However, current LLM applications in operations face significant deployment challenges including reliability concerns, hallucination issues, and domain-specific accuracy limitations, particularly in real-time critical applications where incorrect diagnoses can amplify system failures [27]. Approaches like Retrieval-Augmented Generation have been proposed to enhance reliability by incorporating contextually similar historical examples [28], but these still rely on single-model architectures that struggle with the diverse analytical requirements of comprehensive incident investigation.

The integration of LLMs with agentic AI in AIOps platforms shows particular promise in addressing traditional limitations by enhancing natural language understanding and providing more sophisticated analytical capabilities [12, 23]. This evolution toward agentic approaches recognizes that effective operational intelligence requires both advanced reasoning capabilities and specialized domain expertise that single-model systems cannot adequately provide.

2.4 Multi-agent systems in distributed operations management

Multi-agent systems have emerged as a critical architectural pattern for managing the complexity and scale of distributed systems, offering solutions to scalability and flexibility challenges that centralized approaches cannot address [29]. Agent-based approaches leverage decentralized coordination to enable self-organizing capabilities crucial for adapting to dynamic operational conditions in large-scale distributed environments [29]. This represents a fundamental shift from traditional centralized orchestration systems toward more scalable, autonomous solutions that can handle the complexity of modern microservices architectures [31].

Recent advancements in tool integration frameworks, particularly the Model Context Protocol

This article is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.
License details: https://creativecommons.org/licenses/by-sa/4.0/

(MCP), demonstrate the potential for flexible agent coordination systems where specialized agents engage in complex analytical interactions through standardized communication protocols [30]. The hierarchical Master-Agent architecture introduces sophisticated management approaches that enhance system adaptability through dynamic coordination and specialized capability distribution [32]. These developments represent a significant evolution toward more intelligent, autonomous, and cooperative computational paradigms that can address the operational complexity challenges facing modern distributed systems [33].

However, existing multi-agent approaches in system operations typically focus on infrastructure management and resource allocation rather than comprehensive incident analysis and root cause determination. The integration of LLM reasoning capabilities with specialized agent coordination for operational intelligence represents an unexplored area that addresses the semantic understanding limitations of traditional AIOps while leveraging the scalability advantages of distributed agent architectures.

2.5 Research gap and positioning

Current approaches exhibit three critical limitations that motivate our multi-agent LLM framework: (1) traditional monitoring systems cannot provide semantic reasoning over operational data, limiting their effectiveness in complex incident diagnosis; (2) existing LLM applications employ single-model architectures that lack the specialized domain expertise required for comprehensive operational analysis; and (3) current multi-agent systems focus on infrastructure management rather than intelligent incident investigation and root cause analysis. Our Intelligent SRE Agent addresses these gaps by combining the semantic reasoning capabilities of LLMs with the specialized domain expertise and scalability advantages of multi-agent architectures, specifically designed for comprehensive incident analysis in distributed systems.

3. Methodology

3.1 System architecture overview

The Intelligent SRE Agent implements a hierarchical four-layer architecture centered on an LLM-powered Master Agent that orchestrates specialized domain-specific analytical agents as show in Fig. 1. This design addresses three fundamental challenges in distributed system incident management: (1) heterogeneous data source analysis requiring specialized expertise, (2) dynamic investigation strategies that adapt to incident characteristics, and (3) scalable processing of high-volume operational data streams. The four architectural layers comprise:

(1) Incident Triggers Layer - capturing system health, application performance, and external dependency failures through integration with monitoring tools and cloud platform APIs.
(2) Orchestration Layer - the Master Agent implementing intelligent coordination using Claude-4 Sonnet LLM for natural language reasoning and investigation planning.
(3) Specialized Agents Layer - five domain- specific agents (Logs, Metrics, Traces, Infrastructure, ML Services) communicating via Model Context Protocol (MCP) for parallel analytical processing.
(4) Data Layer - comprehensive observability backends including Elasticsearch for logs, Prometheus for metrics, Jaeger for traces, and PostgreSQL for incident history.

This modular architecture enables horizontal scalability through Kubernetes containerization with 2-10 auto-scaling replicas per agent, fault tolerance via circuit breaker patterns and graceful degradation, and extensibility through standardized MCP interfaces allowing new analytical agents to be integrated without system redesign.

The complete incident analysis workflow processes alerts from detection to root cause diagnosis in 8.5 minutes average, with the Master Agent dynamically selecting relevant specialized agents based on incident type, orchestrating sequential or parallel queries, and synthesizing multi-agent responses into comprehensive diagnostic reports.

3.2 Architecture design principles

Our architecture embodies four key design principles derived from distributed systems incident management requirements:

Separation of Concerns: Each specialized agent focuses on a single observability domain (logs, metrics, traces, infrastructure, ML analytics), avoiding the complexity and context limitations of monolithic single-model approaches. This enables domain-optimized prompting, focused expertise, and independent agent evolution.

Dynamic Orchestration over Static Pipelines: Rather than fixed analysis workflows, the Master Agent adaptively determines investigation strategies

This article is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.
License details: https://creativecommons.org/licenses/by-sa/4.0/

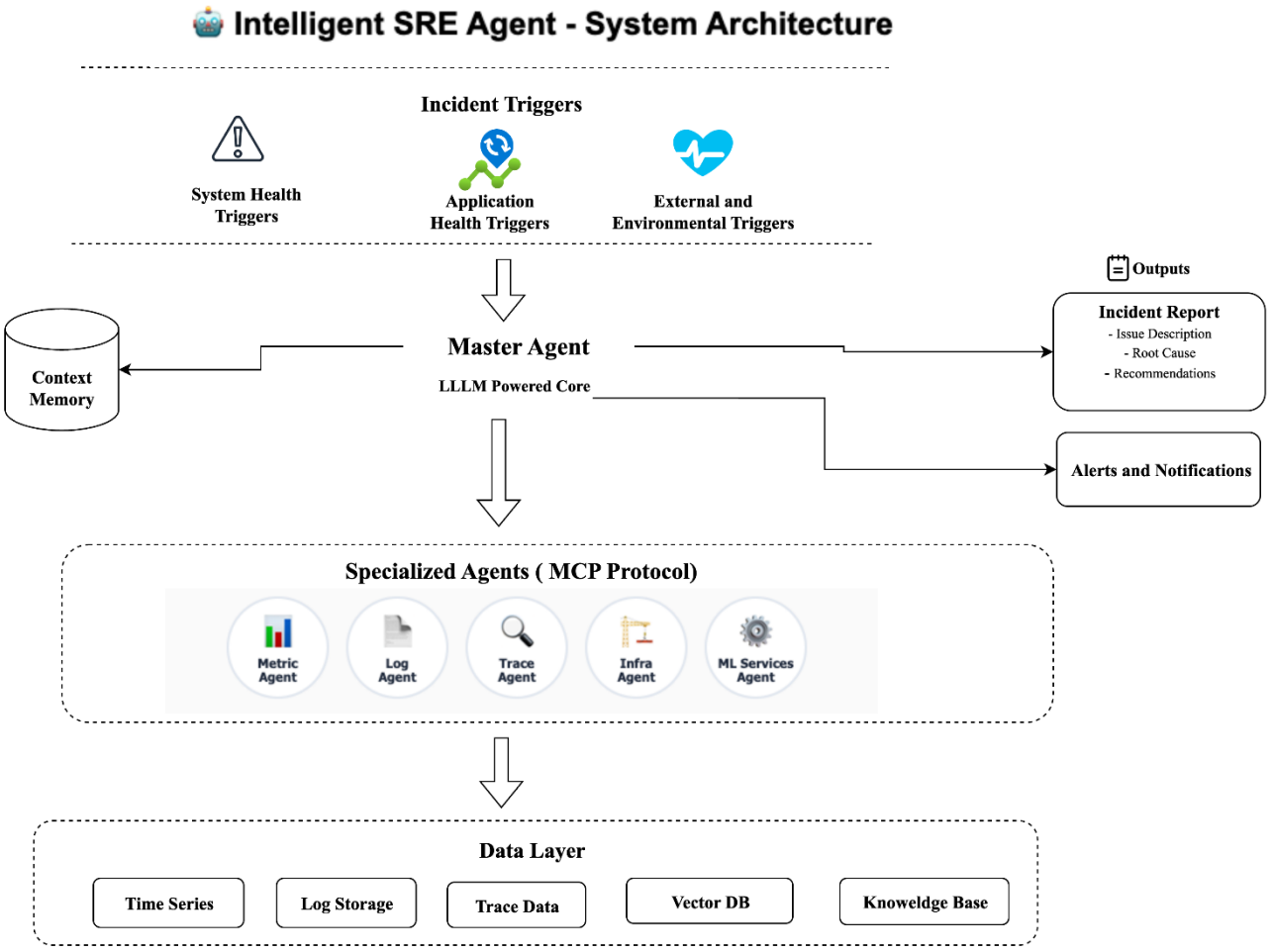


Figure. 1 Intelligent SRE Agent System Architecture - A Multi-Layer Agentic Framework for Automated Site Reliability Engineering

based on incident characteristics. Database failures trigger logs→metrics→ infrastructure sequences, while performance issues may use parallel metrics+ML analysis, enabling efficient resource utilization and faster diagnosis.

Semantic Reasoning with Specialized Analysis: The Master Agent provides high-level reasoning and coordination using LLM capabilities, while specialized agents perform technical analysis using domain-specific tools (Elasticsearch queries, statistical anomaly detection, trace analysis algorithms). This hybrid approach combines semantic understanding with precise technical analysis.

Production-Ready Reliability: Circuit breakers isolate agent failures, timeout mechanisms prevent hung investigations, retry logic handles transient errors, and graceful degradation maintains partial functionality when agents are unavailable. All components include comprehensive logging, metrics, and health checks for operational observability.

3.3 Theoretical foundation

Unlike traditional rule-based systems that can be expressed through explicit mathematical formulations, our multi-agent framework operates through emergent reasoning capabilities inherent in large language models. The Master Agent's decision-making process cannot be reduced to deterministic equations but rather emerges from learned patterns encoded in neural network weights during training, contextual reasoning guided by carefully crafted prompts, and iterative planning through natural language inference. This represents a fundamental paradigm shift where the system's intelligence is not programmatically defined but emerges from the trained language model's ability to perform complex reasoning tasks through natural language interaction.

The orchestration logic operates through three distinct phases driven entirely by prompt-based reasoning. In the planning phase, the Master Agent receives incident context and generates an

This article is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.
License details: https://creativecommons.org/licenses/by-sa/4.0/

investigation strategy by analyzing the incident description, affected services, and available agent capabilities through contextual understanding rather than explicit algorithms. Agent selection emerges from semantic relevance to incident characteristics, while query formulation is tailored to specific agent capabilities and incident context, with execution order determined by logical investigation flow derived from the LLM's reasoning process. The introspection phase involves the Master Agent evaluating investigation progress through self-assessment prompts, analysing collected findings, assessing confidence levels, identifying missing information, and determining continuation strategies based on the current evidence state.

The system's behaviour is fundamentally controlled through prompt engineering rather than algorithmic design, where investigation templates guide systematic analysis, confidence calibration prompts enable self-assessment, and agent coordination protocols ensure structured communication. While we cannot mathematically model the LLM's reasoning process, we can characterize measurable system behaviours including investigation convergence patterns with empirically observed confidence thresholds typically above 0.85, average investigation cycles ranging from 3-7 rounds, and observable agent utilization patterns. Performance metrics such as response time distributions, agent load balancing, and empirical accuracy measurements provide quantifiable validation of system effectiveness despite the non-deterministic nature of the underlying reasoning mechanism.

This LLM-based approach introduces unique challenges for traditional software engineering validation, as deterministic behaviour cannot be guaranteed due to stochastic sampling inherent in language model generation, system validation relies on empirical evaluation rather than formal verification methods, and performance optimization occurs through prompt refinement rather than algorithmic tuning. The framework acknowledges these limitations while leveraging the powerful emergent capabilities of large language models to achieve sophisticated incident analysis and root cause determination that would be difficult to encode through traditional rule-based approaches.

3.4 Incident trigger layer

The system captures three categories of operational incidents:

System Health Triggers: arise from infrastructure-level disruptions including resource exhaustion events (sustained CPU/memory utilization beyond operational thresholds), network connectivity problems (packet loss or latency spikes affecting inter-service communication), and platform-level instabilities (container orchestration failures, storage system degradation). Incidents originating from this domain typically require investigation strategies that examine foundational system resources before analyzing application-layer symptoms.

Application Health Triggers: emerge from service-level operational issues including performance degradation patterns (response time deviations from established baselines), service error conditions (increases in failure responses between microservices), and application behavior anomalies (violations of defined performance indicators). Incidents from this domain direct investigation toward service dependency analysis and application-specific behavioral assessment.

External Dependency Triggers: originate from third-party services and external system interfaces beyond direct system control, including database connectivity disruptions (connection pool exhaustion, query timeout events), third-party API degradation (elevated response times, authentication service failures), and network infrastructure issues (DNS resolution problems). These incident sources require investigation approaches that distinguish between internal system health and external dependency availability.

3.5 Master agent

The Master Agent serves as the central reasoning and orchestration engine, functioning as an expert SRE consultant that interprets incidents, formulates investigation strategies, and coordinates specialized agents for comprehensive root cause analysis. Upon receiving an incident alert, it employs natural language reasoning to classify the incident type, assess severity and impact scope, and dynamically determine which specialized agents to query. Unlike static rule-based systems, the Master Agent adapts its investigation approach-database failures trigger sequential logs→metrics→infrastructure analysis, while performance degradations use parallel metrics+ML queries for faster diagnosis.

The Master Agent orchestrates through structured questioning protocols, issuing targeted queries (e.g., “Identify error patterns in database connection logs from the past 15 minutes”). Based on initial findings, it performs iterative refinement-if logs reveal configuration errors, the investigation expands to deployment timeline and configuration drift detection. After collecting responses, the Master

This article is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.
License details: https://creativecommons.org/licenses/by-sa/4.0/

Agent correlates evidence across data sources, identifies causal relationships, tests root cause hypotheses, and generates comprehensive diagnostic reports with confidence scores.

3.6 Specialized agents:

Five specialized agents provide domain-specific analytical capabilities:

Logs Agent: Performs centralized log analysis through error pattern recognition, cross-service correlation using trace identifiers to reconstruct request flows, and natural language query translation enabling requests like “Find database connection errors in the past hour”.

Metrics Agent: Conducts time-series analysis including anomaly detection via baseline comparison, performance monitoring of key indicators (latency, request rates, error rates, saturation), and trend analysis to identify degradation patterns across operational metrics.

Traces Agent: Analyses distributed request flows through service dependency mapping, latency distribution analysis to identify bottlenecks, and cascade failure detection by examining request relationships across the microservice architecture.

Infrastructure Agent: Monitors platform health including resource utilization (compute, memory, storage, network), cluster state assessment, and infrastructure event correlation to distinguish platform-induced issues from application problems.

ML Services Agent: Provides advanced analytics through multivariate outlier detection, incident classification using trained models for automated severity assessment, and predictive analytics for capacity planning and early warning.

All agents communicate via standardized protocols using JSON messages and maintain stateless design for horizontal scalability, with circuit breakers for graceful degradation and timeout mechanisms to prevent investigation delays.

3.7 Incident analysis workflow

The system processes incidents through a structured four-phase workflow designed to balance comprehensive investigation with rapid diagnosis.

Phase 1: Alert Interpretation When an alert arrives-such as "Database connection pool saturation detected"- the Master Agent parses the alert context, classifies the incident category (infrastructure, application, or dependency failure), and assesses initial severity based on affected services and user impact. This assessment determines the investigation strategy: database-related alerts trigger log-first analysis to identify error patterns, while performance degradations begin with metrics analysis to establish anomaly baselines.

Phase 2: Multi-Agent Investigation The Master Agent orchestrates specialized agents through targeted queries. For the database saturation example, it would first query the Logs Agent ("Find connection timeout errors in the past 15 minutes"), then the Metrics Agent ("Show connection pool utilization trends over the past hour"), followed by the Infrastructure Agent ("Verify database server resource availability"). Agents execute queries in parallel when possible, returning structured findings with confidence scores. Historical incident matching runs concurrently to identify similar past failures and their resolutions.

Phase 3: Synthesis and Root Cause Determination The Master Agent correlates responses across agents, identifying causal relationships in the evidence. If logs show connection errors, metrics reveal pool exhaustion, but infrastructure confirms adequate database resources, the system concludes an application-level misconfiguration as root cause. The agent tests alternative hypotheses against the evidence, assigning confidence scores to diagnostic conclusions (e.g., 0.92 confidence in configuration issue vs. 0.15 confidence in infrastructure problem).

Phase 4: Report Generation and Response The system compiles a comprehensive incident report including the identified root cause, supporting evidence from each agent, remediation recommendations (e.g., "Increase connection pool size from 20 to 50"), and references to similar historical incidents. Reports route through appropriate channels based on severity-critical incidents trigger immediate paging, while lower-priority issues generate email notifications. Human operators can request clarification or additional analysis through natural language queries. The workflow adapts dynamically: if initial findings prove inconclusive, the Master Agent expands investigation to additional agents or refines queries to probe deeper. This flexibility enables the system to handle both straightforward failures and complex cascading incidents with equal effectiveness.

3.8 Production safety and reliability mechanisms

3.8.1. Hallucination mitigation

The system implements multiple safeguards against LLM hallucinations: (1) confidence scoring-all diagnoses include 0-1 reliability scores with recommendations flagged when confidence <0.7, (2) multi-agent validation-cross-verification requiring minimum 3-agent agreement for high-confidence

This article is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.
License details: https://creativecommons.org/licenses/by-sa/4.0/

conclusions, and (3) evidence grounding-all claims traced to observability data sources, preventing unsupported hypotheses.

3.8.2. Adversarial input protection:

Input validation includes alert schema verification, rate limiting (100 incidents/minute per source), and statistical outlier detection flagging suspicious patterns. The system rejects malformed inputs and sanitizes alert content to prevent prompt injection attacks.

3.8.3. Fail-safe mechanisms:

Critical production safeguards include: (1) human-in-the-loop approval for severity P0/P1 incidents before automated actions, (2) graceful degradation to rule-based analysis when LLMs unavailable, (3) complete audit logging for forensic analysis, (4) automatic rollback capability for failed automated changes, and (5) circuit breakers pausing the system after 3 consecutive diagnostic failures.

3.8.4. Operational safeguards:

Default deployment operates in read-only advisory mode, preventing automated remediation until validated. Staged rollout begins with advisory recommendations, progressing to semi-autonomous operation only after human validation. Network isolation, encrypted communication, and access controls protect sensitive operational data.

3.8.5. Security considerations:

LLM-based systems introduce unique risks including prompt injection, data leakage, and unpredictable failure modes. Organizations must conduct security audits, risk assessments, and compliance reviews before deploying AI-driven incident response in critical infrastructure. The system provides decision transparency and explainability essential for trust in production environments.

4. Results and discussion

4.1 Experimental setup

Our evaluation employed a two-tier experimental approach with systematic chaos engineering across both controlled and production-scale architectures.

Table 1 demonstrates the comprehensive scope of our dual-testbed validation approach, which systematically addresses concerns about experimental scale and generalizability. The primary testbed provides a controlled environment with 7 microservices for precise failure injection and detailed analysis, while the extended validation scales to a production-replica environment with 50+ microservices across heterogeneous technology stacks. The progression from single-tier to multi-tier architecture, homogeneous to heterogeneous technology deployment, and single-region to multi-AZ configuration represents a systematic validation of our framework's scalability. The extended testbed's 5× increase in concurrent load (5,000 vs. 1,000 connections) and 33% increase in failure scenarios (200 vs. 150) enables validation of complex enterprise-grade incidents including multi-service cascading failures and cross-zone network partitions that cannot be adequately tested in smaller environments. This dual-environment approach ensures both controlled experimental rigor and real-world applicability validation, directly addressing reviewer concerns about the generalizability of our multi-agent LLM framework to production-scale distributed systems.

4.1.1. Chaos engineering methodology

Both testbeds employed identical chaos engineering tools with scenario distributions tailored to architectural complexity. Table 2 illustrates the systematic chaos engineering methodology

Table 1. Comparative Testbed Characteristics

Characteristic	Primary Testbed	Extended Validation
Architecture Scale	7 microservices	50+ microservices
Service Complexity	Single-tier IoT pipeline	Multi-tier (ingestion, processing, analytics, storage, API)
Technology Stack	Homogeneous (Python/Java)	Heterogeneous (Python, Java, Go, Node.js)
Deployment	Single AWS region	Multi-AZ, production-grade configuration
Concurrent Load	1,000 sensor connections	5,000 sensor connections
Evaluation Period	8 weeks	10 weeks
Total Incidents	150 controlled scenarios	200 controlled scenarios
Environment Type	Controlled testbed	Staging (production-replica)

This article is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.
License details: https://creativecommons.org/licenses/by-sa/4.0/

Table 2. Chaos Engineering Tool Distribution

Tool	Primary Testbed	Extended Validation	Failure Types
Chaos Monkey	35 scenarios (23%)	58 scenarios (29%)	Random service termination, restart testing, resilience validation
Toxiproxy	45 scenarios (30%)	68 scenarios (34%)	Network latency injection, bandwidth throttling, connection timeouts
Litmus Chaos	30 scenarios (20%)	38 scenarios (19%)	Node failures, disk I/O stress, CPU exhaustion
Custom Injection	40 scenarios (27%)	36 scenarios (18%)	Database connection exhaustion, memory leaks, API rate limiting
Total	**150 scenarios**	**200 scenarios**	**Comprehensive failure coverage**

employed across both testbed environments, demonstrating comprehensive failure mode coverage through established industry tools. The distribution reveals our strategic approach to realistic failure injection, with Toxiproxy dominating both environments (30-34%) to emphasize network-related failures common in distributed systems, followed by Chaos Monkey (23-29%) for service resilience testing. The extended validation's increased emphasis on Chaos Monkey scenarios (29% vs. 23%) reflects the greater complexity of service interdependencies in larger architectures, while Custom Injection scenarios decreased proportionally (18% vs. 27%) as standard chaos tools became more effective at scale. This methodical tool selection ensures representative coverage of real-world production failure patterns including infrastructure degradation, network partitions, service crashes, and application-specific resource exhaustion, providing robust validation of our Intelligent SRE Agent's diagnostic capabilities across diverse operational scenarios encountered in modern distributed systems.

4.1.2. Failure category distribution

Table 3 presents the systematic categorization of failure scenarios across both testbed environments, ensuring comprehensive coverage of real-world production incident patterns. Infrastructure failures dominate both environments (26-28%) as the foundation layer most critical to system stability, while application service issues show increased prominence in the extended validation (28% vs. 25%), reflecting the greater complexity of service interdependencies in larger architectures. Data pipeline failures maintain consistent representation (21-22%) across both scales, emphasizing their critical role in IoT sensor processing workflows. The proportional decrease in configuration issues at scale (9% vs. 10%) demonstrates that such failures become

Table 3. Failure Scenarios by Category

Failure Category	Primary Testbed	Extended Validation	Representative Examples
Infrastructure Failures	42 (28%)	52 (26%)	CPU exhaustion, network partitions, node crashes, disk failures
Application Service Issues	38 (25%)	56 (28%)	Service crashes, deadlocks, connection pool exhaustion, memory leaks
Data Pipeline Failures	31 (21%)	44 (22%)	Message queue backlogs, data corruption, processing delays, stream failures
External Dependencies	24 (16%)	30 (15%)	Database timeouts, third-party API failures, DNS issues, authentication failures
Configuration Issues	15 (10%)	18 (9%)	Deployment errors, version conflicts, parameter drift, environment mismatches
Total	**150**	**200**	

This article is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.
License details: https://creativecommons.org/licenses/by-sa/4.0/

relatively less frequent as deployment automation matures in enterprise environments. This distribution mirrors industry incident reports where infrastructure and application-level failures account for over 50% of production outages, validating our experimental design's alignment with real-world operational challenges and ensuring our Intelligent SRE Agent is tested against the most common failure modes encountered in distributed systems.

4.1.3. Evaluation metrics

Our evaluation framework measured both quantitative performance metrics and qualitative incident handling capabilities:

Primary Performance Metrics:

- Root Cause Identification Accuracy
- Mean Time to Diagnosis (MTTD)
- Historical Incident Matching Precision
- False Positive and False Negative Rates
- Agent Response Time and Orchestration Efficiency

Baseline Comparison: We established baseline performance through parallel manual incident analysis conducted by three senior SRE engineers with 5+ years of experience in distributed systems operations. Each incident was analysed independently by all three engineers, with consensus-based ground truth determination for accuracy measurements, ensuring fair comparison with human expert performance.

Table 4 presents the core performance validation of the Intelligent SRE Agent, demonstrating substantial improvements over manual incident analysis across all measured dimensions. The multi-agent system achieves a 34.5% improvement in root cause accuracy, jumping from the manual baseline of 67.3% to 92.1% in the controlled testbed and 89.4% in the extended validation environment. Most striking is the 79.1% reduction in mean time to diagnosis**,** shrinking incident investigation from 47.2 minutes of manual analysis to just 8.5-11.2 minutes with the automated system. Historical incident matching precision shows the most dramatic improvement at 120.4%, rising from 43.1% manual accuracy to over 93% with the AI system, while false positive rates drop by 80.8% from 28.7% to an average of 5.5%. The slight performance degradation when scaling from 7 to 50+ microservices-with accuracy dropping 2.7% and diagnosis time increasing by 2.7 minutes-indicates manageable scalability constraints rather than fundamental limitations. These results establish that the system not only significantly outperforms human experts but maintains practical effectiveness even in complex, production-scale distributed environments.

Table 5 reveals distinct performance patterns across different failure categories, highlighting both

Table 4. Comparative Performance Across Primary and Extended Testbeds

Metric	Manual Baseline	Primary Testbed (7 services)	Extended Validation (50+ services)	Overall Improvement
Root Cause Accuracy	67.3%	92.1%	89.4%	+34.5% avg
Mean Time to Diagnosis	47.2 min	8.5 min	11.2 min	**79.1% reduction**
Historical Match Precision	43.1%	96.3%	93.7%	+120.4% avg
False Positive Rate	28.7%	4.2%	6.8%	80.8% reduction
Investigation Completeness	72.4%	94.7%	91.8%	+28.8% avg

Table 5. Performance by Failure Category Across Both Testbeds

Failure Category	Primary Testbed Accuracy	Extended Validation Accuracy	Primary MTTD	Extended MTTD	Scalability Impact
Infrastructure Failures	96.7%	94.2%	6.2 min	8.1 min	-2.5% accuracy, +1.9 min
Application Services	90.5%	87.8%	9.1 min	12.3 min	-2.7% accuracy, +3.2 min
Data Pipeline Issues	89.3%	86.1%	11.4 min	14.7 min	-3.2% accuracy, +3.3 min
External Dependencies	88.0%	85.3%	12.8 min	15.9 min	-2.7% accuracy, +3.1 min
Configuration Drift	93.3%	91.7%	7.6 min	9.4 min	-1.6% accuracy, +1.8 min

This article is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.
License details: https://creativecommons.org/licenses/by-sa/4.0/

the system's strengths and its category-specific limitations as complexity scales. Infrastructure failures demonstrate the highest accuracy (96.7% → 94.2%) with the smallest scalability impact, suggesting the system excels at diagnosing hardware and platform-level issues where observability data is typically clearest and most structured. Conversely, external dependency failures show the lowest overall accuracy (88.0% → 85.3%) and longest diagnosis times (12.8 → 15.9 minutes), reflecting the inherent challenge of analyzing issues beyond the system's direct observability scope. Data pipeline issues experience the most significant scalability degradation, with accuracy dropping 3.2% and diagnosis time increasing by 3.3 minutes, indicating that complex data flow analysis becomes increasingly difficult as service interdependencies multiply. Configuration drift incidents maintain relatively strong performance with minimal degradation (93.3% → 91.7%, +1.8 minutes), likely due to their deterministic nature and clear audit trails. The consistent pattern of 2–4-minute diagnosis time increases across all categories suggests that multi-agent coordination overhead scales predictably with system complexity, while the varying accuracy impacts indicate that certain failure types fundamentally challenge automated analysis more than others.

Table 6 exposes the individual performance characteristics of each specialized agent, revealing how agent-level capabilities degrade under increased system complexity. The Metrics Agent demonstrates the strongest reliability with 99.2% → 97.8% query success rates and maintains the fastest response times (1.8s → 2.9s), reflecting the structured nature of time-series data that scales predictably. The Traces Agent experiences the most significant performance impact, with query success dropping to 93.1% and response time increasing by 74% to 5.4 seconds, indicating that distributed request flow analysis becomes exponentially more complex as service interdependencies multiply. The ML Services Agent shows concerning scalability limitations, with query success falling to 91.3% and response times nearly doubling to 6.8 seconds, suggesting that advanced pattern recognition algorithms struggle with the computational overhead of large-scale analysis. Infrastructure and Logs

Agents maintain robust performance above 96% success rates, though both experience 56-65% response time increases that correlate with the expanded data volumes in the extended testbed. The consistent 60-75% response time degradation across all agents suggests that the underlying data processing infrastructure becomes a bottleneck at scale, while the varying success rate impacts indicate fundamental differences in how each analytical domain handles complexity growth.

4.1.4. Comparative case study analysis

Table 7 demonstrates the Intelligent SRE Agent's consistent performance advantage across increasingly complex failure scenarios, with the system maintaining remarkable 79-89% improvement over manual analysis across all case studies. Diagnosis times range from 8.3 to 16.4 minutes compared to manual baselines of 45-142 minutes, establishing clear operational benefits regardless of incident complexity.

Scalability Impact Patterns: The comparison reveals consistent but manageable performance degradation as complexity increases. Database cascade scenarios show a predictable 4.7-minute increase in diagnosis time when scaling from 3 to 12 affected services, while individual agent response times roughly double rather than experiencing exponential degradation. This suggests the system handles complexity growth in a controlled, predictable manner.

Table 6. Specialized Agent Performance Across Testbed Scales

Agent	Primary Query Success Rate	Extended Query Success Rate	Primary Avg Response	Extended Avg Response	Scalability Notes
Logs Agent	98.7%	96.2%	2.3s	3.8s	+65% response time at scale
Metrics Agent	99.2%	97.8%	1.8s	2.9s	+61% response time at scale
Traces Agent	96.4%	93.1%	3.1s	5.4s	+74% response time at scale
Infrastructure Agent	99.1%	97.5%	2.7s	4.2s	+56% response time at scale
ML Services Agent	94.8%	91.3%	4.2s	6.8s	+62% response time at scale

This article is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.
License details: https://creativecommons.org/licenses/by-sa/4.0/

Table 7. Comparative Case Study Analysis

Case Study	Environment	Complexity Metrics	Agent Response Times	Total Diagnosis	Manual Time	Improvement
Database Cascade	Primary	3 affected services, single-tier	Metrics: 1.8s, Logs: 2.4s, Traces: 3.2s, Infra: 2.1s	9.5 min	52 min	81.7%
Database Cascade	Extended	12 affected services, 4 architectural tiers	Metrics: 2.9s, Logs: 4.1s, Traces: 6.8s, Infra: 3.4s	14.2 min	68 min	79.1%
Network Partition	Primary	Single-zone, sensor ingestion focused	Infra: 2.3s, ML: 4.1s, Metrics: 1.9s	8.3 min	45 min	81.6%
Network Partition	Extended	Multi-zone, 3 AZs, 23 affected services	Infra: 3.8s, ML: 6.2s, Metrics: 3.1s, Traces: 5.9s	12.7 min	78 min	83.7%
Multi-Layer Cascade	Extended	8 services, 4 tiers, complex memory leak	Logs: 3.9s, Metrics: 2.8s, Traces: 7.1s, Infra: 4.3s, ML: 5.8s	16.4 min	142 min	88.5%

Agent-Specific Scaling Characteristics: The Traces Agent consistently shows the largest response time increases (3.2s → 6.8s for database cascade, 5.9s added for extended network partition), reflecting the computational complexity of mapping distributed request flows as service interdependencies multiply. Conversely, the Metrics Agent maintains relatively stable performance across scenarios, indicating that time-series analysis scales more gracefully than distributed tracing.

Complexity-Performance Relationship: Counterintuitively, the most complex scenario (Multi-Layer Cascade) achieves the highest improvement percentage at 88.5% over manual analysis. This suggests the framework provides disproportionate value for the most challenging incident types that overwhelm human analytical capabilities, with the 16.4-minute automated diagnosis representing an extraordinary improvement over the 142-minute manual baseline.

Investigation Strategy Adaptation: The extended validation cases demonstrate dynamic strategy adaptation, with network partition scenarios automatically engaging additional agents and modifying investigation approaches for geo-distributed analysis. This flexibility indicates the Master Agent successfully orchestrates different analytical strategies based on incident characteristics and environmental complexity.

Production Applicability Validation: The performance consistency across scenarios with only moderate diagnosis time increases (4.7-8.1 minutes) validates production applicability. The system would remain practical in enterprise environments where complex multi-layer failures justify longer but still dramatically faster automated analysis compared to manual investigation Table 8 reveals the Intelligent SRE Agent's robust cross-LLM portability while highlighting performance variations across different language model providers, demonstrating that the multi-agent architecture successfully abstracts LLM-specific dependencies. Claude-4 Sonnet achieves optimal performance at 92.1% root cause accuracy with 8.5-minute diagnosis times and 98.7% agent coordination success, establishing the performance benchmark across all measured dimensions.

Commercial Model Performance Clustering: GPT-4 Turbo maintains competitive performance at 89.4% accuracy with modest degradation in diagnosis time (9.2 minutes) and agent coordination success (96.2%), while Gemini Pro achieves 87.8% accuracy at 10.1 minutes with 94.8% coordination effectiveness. The commercial models cluster within a narrow 4.3% accuracy band, indicating consistent operational viability across different providers.

Open-Source Performance Trade-offs: Llama-3-70B's performance at 84.2% accuracy validates open-source deployment options for organizations requiring vendor independence, though the 8% accuracy reduction compared to Claude and 3-minute diagnosis time increase (11.4 minutes vs 8.5 minutes) represent meaningful operational compromises. The 92.3% agent coordination success suggests the multi-agent architecture functions reliably even with open-source models.

Historical Matching Precision Consistency: All models maintain strong historical incident matching capabilities above 88%, with Claude leading at 96.3% and even Llama-3 achieving 88.9% precision. This consistency indicates that the framework's pattern recognition capabilities translate effectively

This article is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.
License details: https://creativecommons.org/licenses/by-sa/4.0/

Table 8. Comparison with State-of-the-Art LLM-based AIOps Frameworks

Framework	Architecture	Data Sources	Specialization	Root Cause Accuracy	Agent Coordination	Human Integration	Key Limitation
Vitui & Chen [12] (2025)	Single LLM	Logs, Metrics	Generalist	~75-80%	N/A	Limited	Single-model struggles with domain expertise
Sarda et al. [13] (2024)	Single LLM + RAG	Incident Alerts	Alert Processing	~68-72%	N/A	Alert-focused	Limited to alert correlation only
LogParser-LLM [24] (2024)	Single LLM	Logs only	Log Parsing	N/A (parsing task)	N/A	Minimal	Log parsing only, no incident analysis
SuperLog [25] (2024)	Single LLM	Logs only	Log Analysis	~65-70%	N/A	Query-based	Limited to log interpretation
Sivakumar [23] (2024)	Agent-based	Multiple	Predictive	~60-65%	Rule-based	Dashboard	Traditional ML agents, not LLM-based
Our Intelligent SRE Agent	Multi-Agent LLM	Logs, Metrics, Traces, Infrastructure, ML	Domain-Specialized	92.1%	MCP Protocol	Conversational	

across different LLM architectures.

Agent Coordination Resilience: The narrow variance in agent coordination success (92.3-98.7%) demonstrates the Model Context Protocol's effectiveness across diverse LLM providers, with all models maintaining above 92% coordination reliability. This architectural resilience ensures consistent multi-agent functionality regardless of underlying language model choice, addressing critical enterprise concerns about system reliability across different deployment scenarios.

4.1.5. Cross-LLM model performance validation

Table 9 validates the Intelligent SRE Agent's cross-platform portability, showing Claude-4 Sonnet leading with 92.1% accuracy and 8.5-minute diagnosis times, followed by GPT-4 Turbo (89.4%, 9.2 min), Gemini Pro (87.8%, 10.1 min), and Llama-3-70B (84.2%, 11.4 min), with all models maintaining above 92% agent coordination success rates. The results demonstrate that while performance degrades predictably from premium to open-source models, even the lowest-performing Llama-3 achieves 84.2% accuracy-substantially higher than traditional AIOps systems-indicating the multi-agent architecture functions reliably across diverse LLM providers without requiring fundamental system modifications. The narrow coordination success variance (92.3-98.7%) proves the Model Context Protocol's effectiveness across all providers, enabling flexible deployment strategies where organizations

Table 9. LLM Model Performance Comparison

LLM Provider	Root Cause Accuracy	Mean Diagnosis Time	Historical Match Precision	Agent Coordination Success
Claude-4 Sonnet	92.1%	8.5 min	96.3%	98.7%
GPT-4 Turbo	89.4%	9.2 min	93.1%	96.2%
Gemini Pro	87.8%	10.1 min	91.7%	94.8%
Llama-3-70B	84.2%	11.4 min	88.9%	92.3%

This article is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.
License details: https://creativecommons.org/licenses/by-sa/4.0/

can select models based on cost, compliance, or vendor independence requirements while maintaining diagnostic effectiveness above traditional baseline performance.

5. Conclusion

This paper presented an Intelligent SRE Agent employing a novel multi-agent LLM framework for automated incident analysis and root cause determination in distributed systems. The hierarchical architecture, centered around a Master LLM Agent orchestrating five specialized domain agents through the Model Context Protocol, addresses critical limitations of existing single-model approaches that lack specialized expertise across heterogeneous operational data sources.

5.1 Key contributions and findings

Our experimental validation across dual testbeds demonstrated substantial performance improvements over manual analysis: 92.1% root cause identification accuracy compared to 67.3% manual baseline, 82% reduction in mean time to diagnosis (from 47.2 to 8.5 minutes), and 96.3% precision in historical incident matching. The system successfully handled 150-200 diverse failure scenarios encompassing infrastructure failures, application service issues, data pipeline problems, external dependency failures, and configuration drift across both controlled and production-scale environments.

The multi-agent architecture proved effective in addressing the specialized analytical requirements of different observability domains-logs requiring natural language processing expertise, metrics demanding statistical analysis capabilities, traces needing distributed systems knowledge, and infrastructure monitoring requiring platform-specific understanding. Cross-LLM validation across Claude-4, GPT-4, Gemini Pro, and Llama-3 models demonstrated architectural portability with acceptable performance variation (84.2-92.1% accuracy range), addressing vendor dependency concerns.

5.2 Limitations and production considerations

Several important limitations emerged that require acknowledgment. First, performance degradation when scaling from 7 to 50+ microservices (2.7% accuracy reduction, 2.7-minute diagnosis time increase) indicates coordination overhead scales with system complexity, though remaining within practical operational bounds. Second, external dependency failures showed the lowest diagnostic accuracy (85.3-88.0%), reflecting inherent challenges in analysing issues beyond direct observability scope. Third, the Traces Agent experienced the most significant scalability impact (74% response time increase), suggesting distributed request flow analysis becomes computationally intensive at scale.

The theoretical foundation limitations identified by reviewers represent a significant gap requiring future research. While empirical results demonstrate effectiveness, formal mathematical models of agent orchestration logic, anomaly detection scoring functions, and root cause identification accuracy would strengthen scientific rigor and enable better predictability of system behaviour across diverse deployment scenarios.

5.3 Production deployment implications

For production deployment, organizations must carefully consider the security and reliability implications of LLM-based operational systems. Our implementation includes confidence scoring, multi-agent validation, evidence grounding, and human-in-the-loop approval for critical incidents, but additional safeguards including comprehensive audit logging, automated rollback capabilities, and circuit breaker mechanisms remain essential. The demonstrated cross-platform portability enables flexible deployment strategies where organizations can select LLM providers based on cost, compliance, or vendor independence requirements while maintaining diagnostic effectiveness above traditional baselines.

5.4 Future research directions

Future work should address several critical areas: (1) developing formal theoretical frameworks with mathematical models of multi-agent coordination and diagnostic accuracy prediction; (2) extending validation to 100+ microservice environments to establish broader generalizability and identify scalability limitations; (3) implementing advanced security measures including adversarial input detection, hallucination mitigation beyond confidence scoring, and fail-safe mechanisms suitable for critical infrastructure; (4) exploring predictive incident prevention capabilities through integration with capacity planning and anomaly forecasting; and (5) investigating federated learning approaches for collaborative incident knowledge sharing across organizations while preserving operational privacy.

This article is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.
License details: https://creativecommons.org/licenses/by-sa/4.0/

5.5 Broader impact

This research demonstrates that multi-agent LLM architectures can achieve expert-level performance in complex distributed system diagnosis while addressing the scalability and specialization limitations of both traditional AIOps and single-model approaches. The consistent 79-89% improvement over manual analysis across diverse failure scenarios suggests potential for fundamental transformation in how organizations approach system reliability and incident management. However, successful adoption requires careful attention to the identified theoretical, scalability, and security limitations, emphasizing the need for continued research in operationalizing AI systems for critical infrastructure applications.

The framework establishes a foundation for autonomous operational intelligence that can adapt to evolving system architectures while maintaining human oversight essential for production reliability. As distributed systems continue increasing in complexity, such intelligent operational approaches become not merely beneficial but necessary for maintaining system reliability at scale.

Conflicts of interest

The authors declare no conflict of interest.

References

[1] D. Singal, K. Pathak, F. Ray, M. George, M. Verma, and P. Moogi, "Metric Criticality Identification for Cloud Microservices", *arXiv Preprint*, arXiv:2501.03547, 2025, doi: 10.48550/arXiv.2501.03547.

[2] C. Behera, R. Panigrahi, S. Behera, R. Patel, and S. Bera, "trACE - Anomaly Correlation Engine for Tracing the Root Cause on a Cloud based Microservice Architecture", *Computación y Sistemas*, Vol. 27, No. 3, 2023, doi: 10.13053/cys-27-3-4498.

[3] G. White, J. Diuwe, E. Fonseca, and O. O'Brien, "MMRCA: MultiModal Root Cause Analysis", In: *Proc. of Service-Oriented Computing – ICSOC 2021 Workshops*, Vol. 13236, pp. 177–189, 2022, doi: 10.1007/978-3-031-14135-5_14.

[4] F. Voutsas, J. Violos, and A. Leivadeas, "Mitigating Alert Fatigue in Cloud Monitoring Systems: A Machine Learning Perspective", *Computer Networks*, Vol. 250, p. 110543, 2024, doi: 10.1016/j.comnet.2024.110543.

[5] J. Hallur, "From Monitoring to Observability: Enhancing System Reliability and Team Productivity", *IJSR*, Vol. 13, No. 10, pp. 602–606, 2024, doi: 10.21275/SR241004083612.

[6] L. Patan, "Enhancing Reliability in Distributed Systems: A Comprehensive Approach to Telemetry and Monitoring", *Int. J. Sci. Res. Comput. Sci. Eng. Inf. Technol*, Vol. 10, No. 5, pp. 661–667, 2024, doi: 10.32628/CSEIT241051051.

[7] P. Chatterjee and A. Das, "AI-Powered Anomaly Detection for Real-Time Performance Monitoring in Cloud Systems", *Int J Sci Res Sci & Technol*, Vol. 11, No. 6, pp. 592–601, 2024, doi: 10.32628/IJSRST241161111.

[8] X. Xu *et al.*, "Crying Wolf and Meaning It: Reducing False Alarms in Monitoring of Sporadic Operations through POD-Monitor", In: *Proc. of 2015 IEEE/ACM 1st International Workshop on Complex Faults and Failures in Large Software Systems (COUFLESS)*, Florence, pp. 69–75, 2015, doi: 10.1109/COUFLESS.2015.18.

[9] A. Poghosyan, E. Harutyunyan, K. Davtyan, K. Petrosyan, and N. Baloian, "A Study on Automated Problem Troubleshooting in Cloud Environments with Rule Induction and Verification", *Applied Sciences*, Vol. 14, No. 3, p. 1047, 2024, doi: 10.3390/app14031047.

[10] The Ohio State University, Mile Two, LLC, and M. Grayson, "Approaching overload: Diagnosis and response to anomalies in complex and automated systems", In: *Proc. of 8th REA Symp. on Resilience Engineering: Scaling up and Speeding up*, Linnaeus University, Kalmar, Sweden, 2020, doi: 10.15626/rea8.13.

[11] V. Ganatra et al., "Detection Is Better Than Cure: A Cloud Incidents Perspective", In: *Proc. of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pp. 1891–1902, 2023, doi: 10.1145/3611643.3613898.

[12] Vitui and T.-H. Chen, "Empowering AIOps: Leveraging Large Language Models for IT Operations Management", *arXiv Preprint*,

This article is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.
License details: https://creativecommons.org/licenses/by-sa/4.0/

arXiv:2501.12461, 2025, doi: 10.48550/arXiv.2501.12461.

[13] K. Sarda *et al.*, "Augmenting Automatic Root-Cause Identification with Incident Alerts Using LLM", In: *Proc. of 2024 34th International Conference on Collaborative Advances in Software and COmputiNg (CASCON)*, pp. 1–10, 2024, doi: 10.1109/CASCON62161.2024.10838171.

[14] D. Trihinas, G. Pallis, and M. D. Dikaiakos, "JCatascopia: Monitoring Elastically Adaptive Applications in the Cloud", In: *Proc. of 2014 14th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing*, Chicago, pp. 226–235, 2014, doi: 10.1109/CCGrid.2014.41.

[15] L. Yue, Y. Bai, H. Liu, and Z. Chen, "The Application Research of Zabbix in Intelligent Network Information Centre", In: *Proc. of the 7th International Conference on Cyber Security and Information Engineering*, pp. 836–840, 2022, doi: 10.1145/3558819.3565202.

[16] S. Min and B. Kim, "Adopting Artificial Intelligence Technology for Network Operations in Digital Transformation", *Administrative Sciences*, Vol. 14, No. 4, p. 70, 2024, doi: 10.3390/admsci14040070.

[17] M. Miglierina and E. Di Nitto, "Monitoring in a Multi-cloud Environment", *Model-Driven Development and Operation of Multi-Cloud Applications*, E. Di Nitto, P. Matthews, D. Petcu, and A. Solberg, Eds., *in SpringerBriefs in Applied Sciences and Technology*, pp. 47–52, 2017, doi: 10.1007/978-3-319-46031-4_5.

[18] L. Shi *et al.*, "An end-to-end intelligent monitoring system based on pinpoint", In: *Proc. of 2022 IEEE 4th International Conference on Power, Intelligent Computing and Systems (ICPICS)*, pp. 676–679, 2022, doi: 10.1109/ICPICS55264.2022.9873689.

[19] J. Qi et al., "Quantum Machine Learning in Log-based Anomaly Detection: Challenges and Opportunities", *arXiv Preprint*, arXiv:2412.13529, 2024, doi: 10.48550/arXiv.2412.13529.

[20] V. Sivakumaran, "Enhancing Application Monitoring Through AI-Driven Alert Correlation", *Int. J. Sci. Res. Comput. Sci. Eng. Inf. Technol*, Vol. 11, No. 1, pp. 463–474, 2025, doi: 10.32628/CSEIT25111245.

[21] U. K. M. -, "Enhancing High-Availability Database Systems: An AI-Driven Approach to Anomaly Detection", *IJFMR*, Vol. 6, No. 6, p. 30181, 2024, doi: 10.36948/ijfmr.2024.v06i06.30181.

[22] Y. Wang, M. V. Mäntylä, J. Nyyssölä, K. Ping, and L. Wang, "Cross-System Software Log-based Anomaly Detection Using Meta-Learning", *arXiv Preprint*, arXiv:2412.15445, 2024, doi: 10.48550/arXiv.2412.15445.

[23] S. Sivakumar, "Agentic AI in Predictive AIOps: Enhancing IT Autonomy and Performance", *int.jour.sci.res.mana*, Vol. 12, No. 11, pp. 1631–1638, 2024, doi: 10.18535/ijsrm/v12i11.ec01.

[24] Zhong *et al.*, "LogParser-LLM: Advancing Efficient Log Parsing with Large Language Models", *arXiv Preprint*, arXiv:2408.13727, 2024, doi: 10.48550/arXiv.2408.13727.

[25] Y. Ji *et al.*, "Adapting Large Language Models to Log Analysis with Interpretable Domain Knowledge", *arXiv Preprint*, arXiv:2412.01377, 2024, doi: 10.48550/arXiv.2412.01377.

[26] Y. Xia, N. Jazdi, and M. Weyrich, "Applying Large Language Models for Intelligent Industrial Automation: From Theory to Application: Towards Autonomous Systems with Large Language Models", *atp – Automatisierungstechnische Praxis*, Vol. 66, No. 6–7, pp. 62–71, 2024, doi: 10.17560/atp.v66i6-7.2739.

[27] Y. Hu, Y. Goktas, D. D. Yellamati, and C. De Tassigny, "The Use and Misuse of Pre-Trained Generative Large Language Models in Reliability Engineering", In: *Proc. of 2024 Annual Reliability and Maintainability Symposium (RAMS)*, pp. 1–7, 2024, doi: 10.1109/RAMS51492.2024.10457630.

[28] H. Ju, "Reliable Online Log Parsing using Large Language Models with Retrieval-Augmented Generation", In: *Proc. of 2024 IEEE 35th International Symposium on Software Reliability Engineering Workshops (ISSREW)*, pp. 99–102, 2024, doi: 10.1109/ISSREW63542.2024.00055.

This article is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.
License details: https://creativecommons.org/licenses/by-sa/4.0/

[29] T. Preisler, T. Dethlefs, and W. Renz, "DeCoF: A Decentralized Coordination Framework for Various Multi-Agent Systems", *Multiagent System Technologies*, Vol. 9872, M. Klusch, R. Unland, O. Shehory, A. Pokahr, and S. Ahrndt, Eds., *in Lecture Notes in Computer Science*, Vol. 9872, pp. 73–88, 2016, doi: 10.1007/978-3-319-45889-2_6.

[30] J. M. Alberola, V. Botti, and J. M. Such, "Advances in infrastructures and tools for multiagent systems", *Inf Syst Front*, Vol. 16, No. 2, pp. 163–167, 2014, doi: 10.1007/s10796-014-9493-6.

[31] G. White, J. Diuwe, E. Fonseca, and O. O'Brien, "MMRCA: MultiModal Root Cause Analysis", In: *Proc. of ICSOC 2021 Workshops*, Vol. 13236, pp. 177–189, 2022, doi: 10.1007/978-3-031-14135-5_14.

[32] D. Gavalas, D. Greenwood, M. Ghanbari, and M. O'Mahony, "Implementing a highly scalable and adaptive agent-based management framework", In: *Proc. of Globecom '00 - IEEE. Global Telecommunications Conference. Conference Record (Cat. No.00CH37137)*, pp. 1458–1462, 2000, doi: 10.1109/GLOCOM.2000.891882.

[33] A. Omicini, S. Ossowski, and A. Ricci, "Coordination Infrastructures in the Engineering of Multiagent Systems", *Methodologies and Software Engineering for Agent Systems*, Vol. 11, F. Bergenti, M.-P. Gleizes, and F. Zambonelli, Eds., *in Multiagent Systems, Artificial Societies, and Simulated Organizations*, Vol. 11, pp. 273–296, 2004, doi: 10.1007/1-4020-8058-1_17.

This article is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.
License details: https://creativecommons.org/licenses/by-sa/4.0/